

Battleship AI: Observations of Neural Networks that Efficiently Play Battleship

Brendan Phoebe & Christine Le

University of Arizona – SUnMaRC 2026



Intro to Battleship Board Game

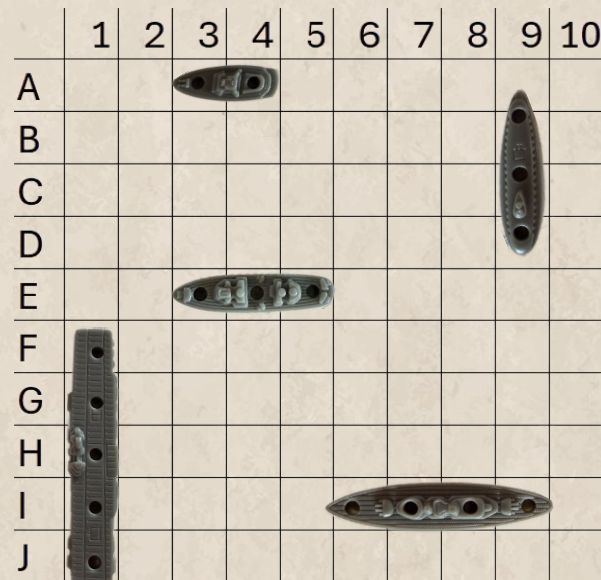
Each player gets a grid and secretly places their 5 ships, of size 5, 4, 3, 3, and 2, then alternate calling shots to locate and sink the opponent's fleet.

"Hit" — the shot lands on an occupied cell.

"Miss" — the shot lands on an empty cell.

A ship is **"sunk"** when all its cells have been hit.

The first player to sink all five opposing ships wins.

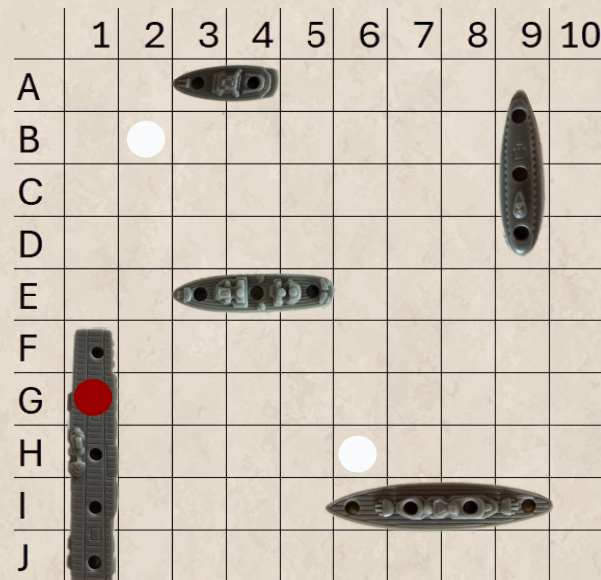
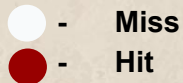


Random Algorithm

Shots are fired at random.

The worst-case scenario:

- Pure random shooting would be about **mean 97 shots**
- With "**hit chasing**" (once a hit is found, use ship killing algorithm), the random average drops to a mean of **67 shots**.



Checkerboard Algorithm

To find the boat with length of 5 shoot diagonally every 5 until boat is found.

This does not work for the smaller boats.

For the boat of length four shoot diagonally by four until boat is found.

	1	2	3	4	5	6	7	8	9	10
A	●		●	●	●		●		●	
B		●		●		●		●		●
C	●		●		●		●		●	
D		●		●		●		●		●
E	●		●		●		●		●	
F		●		●		●		●		●
G	●		●		●		●		●	
H		●		●		●		●		●
I	●		●		●		●		●	
J		●		●		●		●		●

For the two boats of length 3 shoot diagonally until the both boats are found

For the boat of length two, shoot every other until the boat is found.

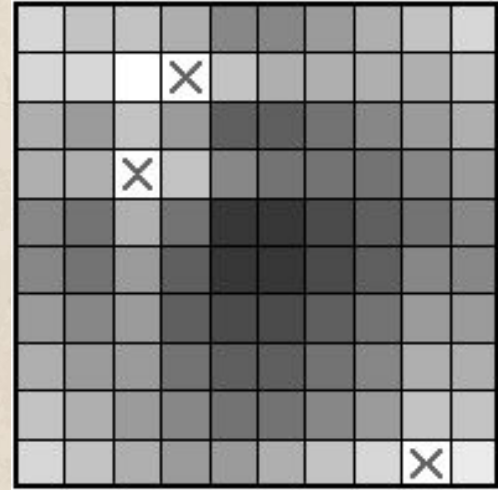
Variations and combinations of these techniques can be used to efficiently find ships.

The Probabilistic Approach

The Probabilistic Approach to Battleship involves counting every possible setup of ships and firing at the cell that is occupied the most often, and repeating for each turn.

This is the most optimal method when playing against random defenses, and results in a median number of turns of 42.

However, this method can be computationally inefficient when used without sampling, as there are up to 30,093,975,536 possible combinations on the first turn.



Source: Berry, Nick. "Battleship."
DataGenetics, Dec. 3, 2011.

A **Neural Network** is a machine learning model that **learns patterns from data** by sending information through **layers of "Neurons."** At the lowest layer Neurons hold information about the data, but **layer by layer the data is transformed into an output.**

The lowest hidden layer tends to understand simple patterns, and each consecutive layer will **find patterns within those patterns.** The "weights" for these transformations from layer to layer can be found through multiple methods, but the technique we will use is called **"neuroevolution."**

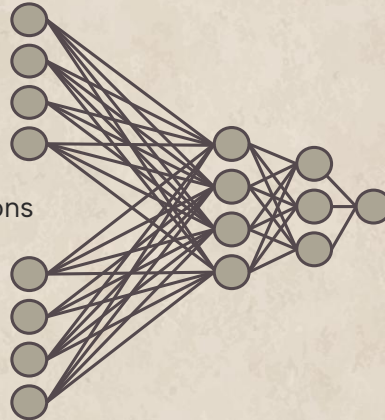
	1	2
A		
B		

=

A1 Hit = 1
A1 Miss = 0
A1 Hit_unsunk = 0
A1 Was_shot = 1

... many many neurons

B2 Hit = 0
B2 Miss = 0
B2 Hit_unsunk = 0
B2 Was_shot = 0



= We should shoot A2

Neuroevolution

Just like biological evolution, neuroevolution uses random **mutation and survival-of-the-fittest** to train better neural networks.

In the first generation, weights are assigned, and the model who happens to do the **best is chosen to have the next generation** be based on their weights, with **small deviations for mutation**. This process is repeated every generation to produce increasingly better NNs.

Candidates: The population of Each Generation

Games Per Candidate: The Games each Candidate plays so we can assess their ability

Generations: How many Generations we will train.

	1	2
A	●	●
B	○	○
WINNER!		

	1	2
A	●	○
B	○	○
LOSER!		

	1	2
A	●	○
B	○	●
LOSER!		

Winners babies

	1	2
A	○	●
B	○	○
LOSER!		

	1	2
A	●	○
B	○	○
LOSER!		

	1	2
A	●	●
B	○	○
WINNER!		

Winners babies

	1	2
A	●	○
B	○	○
LOSER!		

	1	2
A	●	○
B	○	○
LOSER!		

	1	2
A	●	●
B	○	○
WINNER!		

etc.

What Are Our Nodes?

Unknown — 1 if the square has never been shot, otherwise 0.

Miss — 1 if the square is a known miss, otherwise 0.

Hit Unsunk — 1 if the square is a hit on a ship that has not been sunk yet.

Sunk — 1 if the square belongs to a ship that is already sunk.

Was Shot — 1 if the square has been fired at before.

Adjacent Miss Fraction — fraction of the 4 neighboring squares that are misses.

Adjacent Unknown Fraction — fraction of the 4 neighboring squares that are still unknown.

Horizontal Room Fraction — available horizontal space through that square, normalized by board size.

Vertical Room Fraction — available vertical space through that square, normalized by board size.

Best Room Fraction — the larger of the horizontal and vertical room values.

Parity Feature — checkerboard-style parity indicator used while ships larger than length 1 remain.

Center Bias — how close the square is to the center of the board.

Row Position — normalized row location from top to bottom.

Column Position — normalized column location from left to right.

Turn Fraction — how far along the game is, based on the current turn number.

Ships Remaining Fraction — fraction of ships still unsunk.

Remaining Length-2 Fraction — fraction of length-2 ships still alive.

Remaining Length-3 Fraction — fraction of length-3 ships still alive.

Remaining Length-4 Fraction — fraction of length-4 ships still alive.

Remaining Length-5 Fraction — fraction of length-5 ships still alive.

We have

20 – input nodes,

32 – layer 1 hidden nodes,

16 – layer 2 hidden nodes

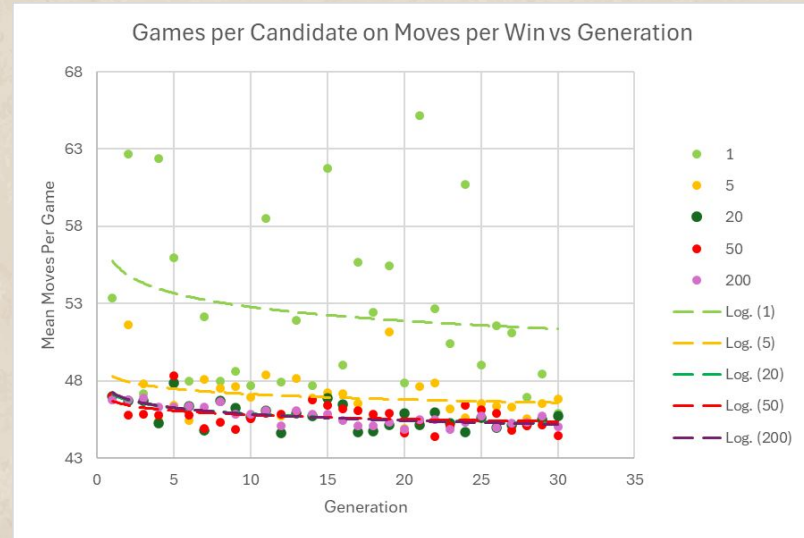
Applied to each cell on the board
every turn

How Many Games Per Candidate?

Too many games? Computationally expensive

Too little games? Can't accurately assess the ability of a model.

We found that, with 50 candidates and 30 generations, the positive effect of games per candidate is negligible after approximately 20 games per candidate.

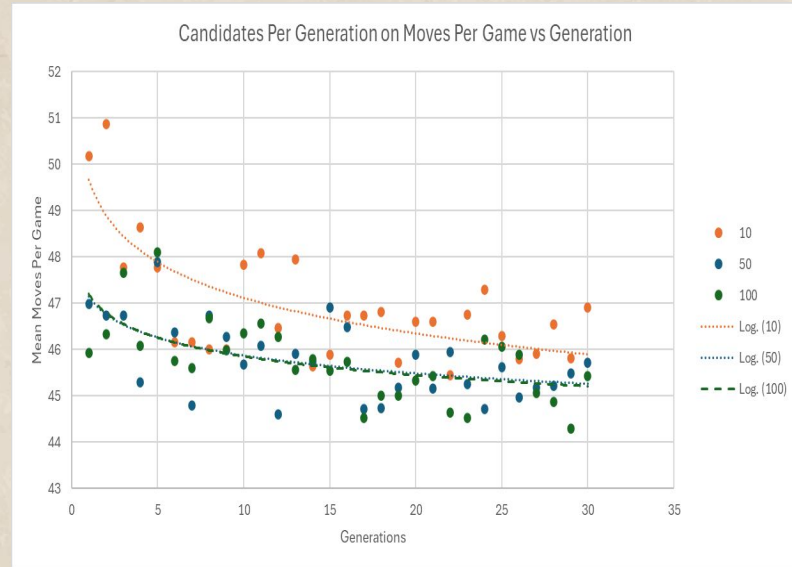


How Many Candidates Per Generation?

The number of candidates per generation determines how many avenues the NN can explore per generation. It allows the NN to avoid falling into a bad strategy and explore many possibilities.

Again, computational expense increases with candidates per generation

As seen on the right, for a relatively simple game like battleship, the difference between 50 and 100 Candidates per generation was negligible.



How Well Can Our NN Perform?

Algorithm	Mean Moves per Game
Random Shots	97 Moves
Random Shots w/ Hit-Chasing Algorithm	67 Moves
Our NN	45.5 Moves
Probabilistic Search	42 Moves

Additionally our NN can also run several hundred games per minute,
As opposed to the heavy computation in probabilistic search.

Thank You!

Follow us on LinkedIn!

CHRISTINE LE

<https://www.linkedin.com/in/christine-le-ab3752369/>

BRENDAN PHOEBE

www.linkedin.com/in/brendan-phoebe

CITATIONS:

Berry, Nick. "Battleship." DataGenetics, Dec. 2011,
<http://www.datagenetics.com/blog/december32011/>.

	1	2	3	4	5	6	7	8	9	10
A	●		●		●		●		●	
B		●		●		●		●		●
C	●		●		●		●		●	
D		●		●		●		●		●
E	●		●		●		●		●	
F		●		●		●		●		●
G	●		●		●		●		●	
H		●		●		●		●		●
I	●		●		●		●		●	
J		●		●		●		●		●